



NAME : HAMDAN ALI

WEEK 5 : Day 24

DAY 24

Create a form with live preview using React state.

Task : Screenshots Of Output

CODEBASE Of the Work

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, including the 'components' folder where 'FormField.jsx' is located. The code editor shows the following code for 'FormField.jsx':

```
2
3 const FormField = ({
4   label,
5   name,
6   type = 'text',
7   value,
8   onChange,
9   error,
10  required,
11  placeholder,
12  maxLength
13 }) => {
14   const isTextarea = type === 'textarea';
15   const InputComponent = isTextarea ? 'textarea' : 'input';
16
17   // Validation icon logic (simple version)
18   // For required fields, if they have a value and no error, show success
```

The terminal output shows the following commands and messages:

```
PS D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship ( HindukushSoft Technologies )\Week5\0
run `npm fund` for details

found 0 vulnerabilities
❖ PS D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship ( HindukushSoft Technologies )\Week5\0
> day24-form-live-preview@1.0.0 dev
> vite

1:59:45 PM [vite] warning: `esbuild` option was specified by "vite:react-babel" plugin. This option is deprecated, please
`optimizeDeps.rollupOptions` / `ssr.optimizeDeps.rollupOptions` is deprecated. Use `optimizeDeps.rolldownOptions` instead
plugin. Set VITE_DEPRECATION_TRACE=1 to see where it is called.
[vite:react-babel] We recommend switching to `@vitejs/plugin-react-oxc` for improved performance. More information at https://vitejs.dev/guide/faq.html#react-oxc
Warning: Invalid input options (1 issue found)
- For the "jsx". Invalid key: Expected never but received "jsx".

VITE v8.2.1 ready in 539 ms

→ Local: http://localhost:5173/
```



Full Task View

Profile Card Builder

Personal Details

Full Name

Jane Doe

Job Title

Frontend Developer

Company Name

Tech Corp

Bio / Tagline

Passionate about building great user experiences...

0 / 120

Contact Information

Email Address

jane@example.com

Phone Number

+1 (555) 000-0000

Website URL

https://janedoe.com

Social Links & Media

Avatar URL

https://example.com/avatar.jpg

GitHub Username

janedoe

LinkedIn Username

in/janedoe

Twitter Handle

@janedoe

Card Customization

Theme Color

Card Layout

Standard

Compact

Detailed

Clear All

??

Your Name

Your Profession



EXPLANATION OF THE PROJECT

DAY 24

Day 24: Form with Live Preview - Technical Documentation

Project: Professional Business Card / Profile Card Builder

Introduction and Approach

Today, I built a real-time Profile Card Builder. My goal was to create a sleek, premium experience where the user can input their professional details and instantly see a beautiful glassmorphism-styled business card generate on the screen. I chose a split-screen layout for desktop, keeping the form easily accessible on the left while the card stays fixed in a stylized preview pane on the right.

Setup and Tools Used

I set up the project from scratch using Vite and React, manually configuring the `package.json`, `vite.config.js`, and HTML template. For styling, I stuck strictly to vanilla CSS using custom properties (CSS variables) to handle theming. I incorporated Google Fonts—Inter for readable body text and Space Grotesk for bold headings—and used layered box-shadows and semi-transparent backgrounds to achieve the glassmorphism effect.

Code Walkthrough

App.jsx (State Management)

The core of the data flow happens in `App.jsx`. I defined an `initialFormData` object and used the `useState` hook to manage it. This centralized state is crucial because it needs to be passed down to both the `ProfileForm` (for inputs) and the `LivePreview` (for rendering). The `handleInputChange` function dynamically updates properties based on the `name` attribute of the input elements, keeping the code DRY.

ProfileForm.jsx (Controlled Inputs and Validation)

This component handles all the user inputs. I implemented simple validation using a `useEffect` hook that watches `formData` and runs regex checks for email, phone, and website formats, populating an `errors` state object. Each input is wrapped in a reusable `FormField` component.



FormField.jsx (Reusable Wrapper)

I abstracted the input structure into a reusable component that handles the label, input/textarea element, error display, and a dynamic character counter (used for the bio section). It conditionally renders success (green checkmark) or error icons based on the validation state passed down from the parent.

LivePreview.jsx (Real-time Rendering)

This component is purely presentational. It takes `formData` as props and uses conditional rendering extensively. For example, if no avatar URL is provided, it falls back to a CSS-styled circle containing the user's initials. Entire sections, like the social links container or the company field, simply don't render if their respective data is missing, keeping the card looking clean at all times. The theme and layout variants are applied simply by appending classes like `theme-ocean` or `layout-compact` dynamically.

Key Learnings

Building this reinforced the power of controlled components in React. Having a single source of truth for the form data makes complex, multi-field UI updates feel effortless. I also learned a lot about organizing CSS custom properties to manage multiple color themes without needing a heavy framework like Tailwind.

Challenges and Solutions

One challenge was handling image loading errors for the avatar. If a user pastes an invalid image URL, the broken image icon looks terrible. I solved this by adding an `onError` event handler directly to the image tag in `LivePreview.jsx` that hides the image and shows the initials placeholder fallback instead. Another challenge was the layout flexibility of the card. I ended up structuring the CSS so that layout classes (Standard, Compact, Detailed) just override specific padding, sizing, and alignment rules of the base card style.

Conclusion

The result is a highly polished, responsive tool. The real-time feedback loop between typing in the form and seeing the card update instantly provides a fantastic user experience.

THANK YOU!